

257 Java String Programs

Using the Stream API

A practical Java 8 reference of focused solutions and examples

How to use this guide. Every numbered item is a focused code fragment intended for a Java method. Use the variables named in the listing (for example, words, text, or sentence) with values appropriate to your program. The guide uses the Java 8 Stream API. Add imports as needed: `java.util.*`, `java.util.function.*`, `java.util.stream.*`, and `java.util.Locale`. Every listing includes a literal sample input and the value it computes. For `HashMap/groupingBy` examples, map entry order is not guaranteed; those samples show the exact entries and explicitly mark their order as variable. The list is comprehensive in practical scope, but it cannot be literally exhaustive because Stream operations can be combined indefinitely.

Contents

1. Basic filtering and mapping (1-25)
2. Character-stream operations (26-54)
3. Word and token processing (55-79)
4. Lists of strings (80-105)
5. Frequency, grouping, and counting (106-135)
6. Matching, reduction, and Optional (136-155)
7. Collectors and map recipes (156-175)
8. Transformations and practical recipes (176-200)
9. Interview checklist gap coverage (201-257)

1. Basic filtering and mapping

1. Keep strings longer than 4 characters

Filter a list by length.

```
List<String> result = words.stream()
    .filter(s -> s.length() > 4)
    .collect(Collectors.toList());
```

Sample input: words = ["java", "streams", "api"]

Sample output: ["streams"]

2. Keep strings starting with a prefix

Select strings that begin with 'pro'.

```
List<String> result = words.stream()
    .filter(s -> s.startsWith("pro"))
    .collect(Collectors.toList());
```

Sample input: words = ["pro", "java", "program"]

Sample output: ["pro", "program"]

3. Keep strings ending with a suffix

Select file names ending in .java.

```
List<String> javaFiles = files.stream()
    .filter(name -> name.endsWith(".java"))
    .collect(Collectors.toList());
```

Sample input: files = ["App.java", "notes.txt", "Readme.md"]

Sample output: ["App.java"]

4. Keep strings containing text

Perform a case-sensitive substring search.

```
List<String> result = words.stream()
    .filter(s -> s.contains("stream"))
    .collect(Collectors.toList());
```

Sample input: words = ["stream", "java"]

Sample output: ["stream"]

5. Remove empty strings

Keep non-empty values only.

```
List<String> result = words.stream()
    .filter(s -> !s.isEmpty())
    .collect(Collectors.toList());
```

Sample input: words = ["", "java"]

Sample output: ["java"]

6. Remove blank strings safely

Trim first, and exclude null or whitespace-only values.

```
List<String> result = words.stream()
    .filter(s -> s != null && !s.trim().isEmpty())
    .collect(Collectors.toList());
```

Sample input: words = [null, " ", "java"]

Sample output: ["java"]

7. Convert all strings to uppercase

Map each value to uppercase using a fixed locale.

```
List<String> result = words.stream()
    .map(s -> s.toUpperCase(Locale.ROOT))
    .collect(Collectors.toList());
```

Sample input: words = ["java", "api"]

Sample output: ["JAVA", "API"]

8. Convert all strings to lowercase

Map each value to lowercase using a fixed locale.

```
List<String> result = words.stream()
    .map(s -> s.toLowerCase(Locale.ROOT))
    .collect(Collectors.toList());
```

Sample input: words = ["JAVA", "API"]

Sample output: ["java", "api"]

9. Trim every string

Normalize leading and trailing spaces.

```
List<String> result = words.stream()
    .map(String::trim)
    .collect(Collectors.toList());
```

Sample input: words = [" java ", "api"]

Sample output: ["java", "api"]

10. Replace spaces with hyphens

Build simple slugs from display text.

```
List<String> slugs = titles.stream()
    .map(s -> s.trim().replaceAll("\\s+", "-"))
    .collect(Collectors.toList());
```

Sample input: titles = ["Java Stream API"]

Sample output: ["Java-Stream-API"]

11. Add a prefix to every string

Decorate each value.

```
List<String> result = words.stream()
    .map(s -> "item-" + s)
    .collect(Collectors.toList());
```

Sample input: words = ["java", "api"]

Sample output: ["item-java", "item-api"]

12. Add a suffix to every string

Append a file extension.

```
List<String> paths = names.stream()
    .map(s -> s + ".txt")
    .collect(Collectors.toList());
```

Sample input: names = ["a", "b"]

Sample output: ["a.txt", "b.txt"]

13. Extract the first character

Take the first letter from non-empty strings.

```
List<String> initials = names.stream()
    .filter(s -> !s.isEmpty())
    .map(s -> s.substring(0, 1))
    .collect(Collectors.toList());
```

Sample input: names = ["Ana", "Bob"]

Sample output: ["A", "B"]

14. Extract the last character

Take the final letter from non-empty strings.

```
List<String> lastChars = words.stream()
    .filter(s -> !s.isEmpty())
    .map(s -> s.substring(s.length() - 1))
    .collect(Collectors.toList());
```

Sample input: words = ["java", "api"]

Sample output: ["a", "i"]

15. Reverse every string

Reverse text with `StringBuilder`.

```
List<String> result = words.stream()
    .map(s -> new StringBuilder(s).reverse().toString())
    .collect(Collectors.toList());
```

Sample input: words = ["java", "api"]

Sample output: ["avaj", "ipa"]

16. Capitalize the first letter

Create title-style words without changing the remainder.

```
List<String> result = words.stream()
    .filter(s -> !s.isEmpty())
    .map(s -> s.substring(0, 1).toUpperCase(Locale.ROOT) + s.substring(1))
    .collect(Collectors.toList());
```

Sample input: words = ["java", "stream"]

Sample output: ["Java", "Stream"]

17. Keep only alphabetic strings

Validate simple letter-only identifiers.

```
List<String> result = values.stream()
    .filter(s -> s.matches("[A-Za-z]+"))
    .collect(Collectors.toList());
```

Sample input: values = ["java", "a1", "API"]

Sample output: ["java", "API"]

18. Keep only numeric strings

Validate positive whole-number text.

```
List<String> result = values.stream()
    .filter(s -> s.matches("\\d+"))
    .collect(Collectors.toList());
```

Sample input: values = ["123", "12a", "007"]

Sample output: ["123", "007"]

19. Keep valid email-shaped strings

Use a lightweight syntax check, not full email validation.

```
List<String> result = emails.stream()
    .filter(s -> s.matches("^^[^@\\s]+@[^@\\s]+\\.^[^@\\s]+$"))
    .collect(Collectors.toList());
```

Sample input: emails = ["ana@mail.com", "bad"]

Sample output: ["ana@mail.com"]

20. Keep strings with an exact length

Find fixed-width values.

```
List<String> result = codes.stream()
    .filter(s -> s.length() == 6)
    .collect(Collectors.toList());
```

Sample input: codes = ["abc", "abcdef", "xy"]

Sample output: ["abcdef"]

21. Keep strings within a length range

Select strings from 3 through 10 characters.

```
List<String> result = words.stream()
    .filter(s -> s.length() >= 3 && s.length() <= 10)
    .collect(Collectors.toList());
```

Sample input: words = ["hi", "java", "streaming"]

Sample output: ["java", "streaming"]

22. Replace vowels in every string

Mask vowels using a regular expression.

```
List<String> result = words.stream()
    .map(s -> s.replaceAll("[aeiouAEIOU]", "*"))
    .collect(Collectors.toList());
```

Sample input: words = ["Java", "API"]

Sample output: ["J*v*", "*P*"]

23. Remove digits from every string

Keep only non-digit characters.

```
List<String> result = values.stream()
    .map(s -> s.replaceAll("\\d", ""))
    .collect(Collectors.toList());
```

Sample input: values = ["a1b2", "java"]

Sample output: ["ab", "java"]

24. Remove punctuation from every string

Retain letters, digits, and whitespace.

```
List<String> result = values.stream()
    .map(s -> s.replaceAll("[^A-Za-z0-9\\s]", ""))
    .collect(Collectors.toList());
```

Sample input: values = ["Hi, Java!"]

Sample output: ["Hi Java"]

25. Normalize internal whitespace

Collapse runs of whitespace to one space.

```
List<String> result = values.stream()
    .map(s -> s.trim().replaceAll("\\s+", " "))
    .collect(Collectors.toList());
```

Sample input: values = [" Java Stream API "]

Sample output: ["Java Stream API"]

2. Character-stream operations

26. Count characters in one string

Turn characters into an IntStream and count them.

```
long count = text.chars()
    .count();
```

Sample input: text = "Java"

Sample output: 4

27. List characters in a string

Convert character codes to one-character strings.

```
List<String> chars = text.chars()
    .mapToObj(c -> String.valueOf((char) c))
    .collect(Collectors.toList());
```

Sample input: text = "abc"

Sample output: ["a", "b", "c"]

28. Count vowels

Count a, e, i, o, and u regardless of case.

```
long vowels = text.toLowerCase(Locale.ROOT).chars()
    .filter(c -> "aeiou".indexOf(c) >= 0)
    .count();
```

Sample input: text = "Education"

Sample output: 5

29. Count consonants

Count letters that are not vowels.

```
long consonants = text.toLowerCase(Locale.ROOT).chars()
    .filter(Character::isLetter)
    .filter(c -> "aeiou".indexOf(c) < 0).count();
```

Sample input: text = "abc123"

Sample output: 2

30. Count digits

Count numeric characters.

```
long digits = text.chars()
    .filter(Character::isDigit)
    .count();
```

Sample input: text = "a1b2"

Sample output: 2

31. Count whitespace characters

Count spaces, tabs, and newlines.

```
long spaces = text.chars()
    .filter(Character::isWhitespace)
    .count();
```

Sample input: text = "a bltc"

Sample output: 2

32. Count uppercase letters

Identify uppercase alphabetic characters.

```
long upper = text.chars()
    .filter(Character::isUpperCase)
    .count();
```

Sample input: text = "JaVA"

Sample output: 3

33. Count lowercase letters

Identify lowercase alphabetic characters.

```
long lower = text.chars()
    .filter(Character::isLowerCase)
    .count();
```

Sample input: text = "JaVA"

Sample output: 1

34. Remove duplicate characters

Keep first occurrence order.

```
String result = text.chars()
    .distinct().mapToObj(c -> String.valueOf((char) c))
    .collect(Collectors.joining());
```

Sample input: text = "programming"

Sample output: "progamin"

35. Sort characters alphabetically

Sort character codes, then join them.

```
String result = text.chars().sorted()
    .mapToObj(c -> String.valueOf((char) c))
    .collect(Collectors.joining());
```

Sample input: text = "cba"

Sample output: "abc"

36. Sort characters in reverse order

Box characters so a reverse comparator can be used.

```
String result = text.chars().boxed()
    .sorted(Comparator.reverseOrder())
    .map(c -> String.valueOf((char) c)).collect(Collectors.joining());
```

Sample input: text = "cba"

Sample output: "cba"

37. Find the first character

Return OptionalInt to handle an empty string.

```
OptionalInt first = text.chars()  
    .findFirst();
```

Sample input: text = "Java"

Sample output: J

38. Find the first digit

Locate the first number in text.

```
OptionalInt firstDigit = text.chars()  
    .filter(Character::isDigit)  
    .findFirst();
```

Sample input: text = "ab2c"

Sample output: 2

39. Find the first uppercase character

Locate the first uppercase letter.

```
OptionalInt firstUpper = text.chars()  
    .filter(Character::isUpperCase)  
    .findFirst();
```

Sample input: text = "javaAPI"

Sample output: A

40. Check whether a string has any digit

Use anyMatch for a boolean answer.

```
boolean hasDigit = text.chars()  
    .anyMatch(Character::isDigit);
```

Sample input: text = "ab2"

Sample output: true

41. Check whether all characters are letters

Use allMatch after deciding empty-string behavior.

```
boolean lettersOnly = !text.isEmpty() && text.chars()  
    .allMatch(Character::isLetter);
```

Sample input: text = "Java"

Sample output: true

42. Check whether no characters are whitespace

Validate a compact token.

```
boolean noWhitespace = text.chars()  
    .noneMatch(Character::isWhitespace);
```

Sample input: text = "JavaAPI"

Sample output: true

43. Sum digit values

Convert each digit character to its numeric value.

```
int sum = text.chars()  
    .filter(Character::isDigit)  
    .map(c -> Character.digit(c, 10)).sum();
```

Sample input: text = "a1b23"

Sample output: 6

44. Extract digits as a string

Keep digit characters in their original order.

```
String digits = text.chars()
    .filter(Character::isDigit)
    .mapToObj(c -> String.valueOf((char) c)).collect(Collectors.joining());
```

Sample input: text = "a1b23"

Sample output: "123"

45. Extract letters as a string

Keep letter characters only.

```
String letters = text.chars()
    .filter(Character::isLetter)
    .mapToObj(c -> String.valueOf((char) c)).collect(Collectors.joining());
```

Sample input: text = "a1b23"

Sample output: "ab"

46. Extract uppercase letters

Keep uppercase letters only.

```
String capitals = text.chars()
    .filter(Character::isUpperCase)
    .mapToObj(c -> String.valueOf((char) c)).collect(Collectors.joining());
```

Sample input: text = "JaVa"

Sample output: "JV"

47. Extract lowercase letters

Keep lowercase letters only.

```
String small = text.chars()
    .filter(Character::isLowerCase)
    .mapToObj(c -> String.valueOf((char) c)).collect(Collectors.joining());
```

Sample input: text = "JaVa"

Sample output: "aa"

48. Replace every digit with

Transform characters conditionally.

```
String masked = text.chars()
    .mapToObj(c -> Character.isDigit(c) ? "#" : String.valueOf((char) c))
    .collect(Collectors.joining());
```

Sample input: text = "a1b23"

Sample output: "a#b##"

49. Toggle character case

Swap uppercase/lowercase characters and keep other characters.

```
String swapped = text.chars().mapToObj(c -> {
    char ch = (char) c;
    return Character.isUpperCase(ch) ? String.valueOf(Character.toLowerCase(ch))
        : Character.isLowerCase(ch) ? String.valueOf(Character.toUpperCase(ch)) : String.valueOf(ch);
}).collect(Collectors.joining());
```

Sample input: text = "JaVa1"

Sample output: "jAvA1"

50. Repeat every character twice

Expand each character with flatMap.

```
String doubled = text.chars()
    .mapToObj(c -> String.valueOf((char) c))
    .flatMap(s -> Stream.of(s, s)).collect(Collectors.joining());
```

Sample input: text = "abc"

Sample output: "aabbcc"

51. Keep characters at even indexes

Stream indexes and map them back to characters.

```
String evenIndex = IntStream.range(0, text.length())
    .filter(i -> i % 2 == 0).mapToObj(i -> String.valueOf(text.charAt(i)))
    .collect(Collectors.joining());
```

Sample input: text = "abcde"

Sample output: "ace"

52. Keep characters at odd indexes

Stream indexes and map them back to characters.

```
String oddIndex = IntStream.range(0, text.length())
    .filter(i -> i % 2 != 0).mapToObj(i -> String.valueOf(text.charAt(i)))
    .collect(Collectors.joining());
```

Sample input: text = "abcde"

Sample output: "bd"

53. Get distinct vowels

Find each vowel once while preserving discovery order.

```
String uniqueVowels = text.toLowerCase(Locale.ROOT).chars()
    .filter(c -> "aeiou".indexOf(c) >= 0).distinct()
    .mapToObj(c -> String.valueOf((char) c)).collect(Collectors.joining());
```

Sample input: text = "Education"

Sample output: "euaio"

54. Find the first repeated character

Group first, then find the first input-order character with count greater than one.

```
Map<Character, Long> counts = text.chars().mapToObj(c -> (char) c)
    .collect(Collectors.groupingBy(Function.identity(), LinkedHashMap::new, Collectors.counting()));
Optional<Character> firstRepeated = text.chars().mapToObj(c -> (char) c)
    .filter(c -> counts.get(c) > 1).findFirst();
```

Sample input: text = "swiss"

Sample output: s

3. Word and token processing

55. Split a sentence into words

Trim before splitting to avoid a leading empty token.

```
List<String> words = Arrays.stream(sentence.trim().split("\\s+"))
    .collect(Collectors.toList());
```

Sample input: sentence = "Java Stream API"

Sample output: ["Java", "Stream", "API"]

56. Count words in a sentence

Count whitespace-separated tokens.

```
long wordCount = Arrays.stream(sentence.trim().split("\\s+"))
    .filter(s -> !s.isEmpty()).count();
```

Sample input: sentence = "Java is easy and Java"

Sample output: 5

57. Find the longest word

Use max with a length comparator.

```
Optional<String> longest = Arrays.stream(sentence.trim().split("\\s+"))
    .max(Comparator.comparingInt(String::length));
```

Sample input: sentence = "Java Stream API"

Sample output: "Stream"

58. Find the shortest word

Use min with a length comparator.

```
Optional<String> shortest = Arrays.stream(sentence.trim().split("\\s+"))
    .min(Comparator.comparingInt(String::length));
```

Sample input: sentence = "Java Stream API"

Sample output: "API"

59. Keep words longer than five letters

Filter a token stream by word length.

```
List<String> longWords = Arrays.stream(sentence.split("\\s+"))
    .filter(w -> w.length() > 5).collect(Collectors.toList());
```

Sample input: sentence = "Java streams api"

Sample output: ["streams"]

60. Uppercase every word

Map sentence tokens to uppercase.

```
String result = Arrays.stream(sentence.split("\\s+"))
    .map(w -> w.toUpperCase(Locale.ROOT)).collect(Collectors.joining(" "));
```

Sample input: sentence = "Java API"

Sample output: "JAVA API"

61. Lowercase every word

Map sentence tokens to lowercase.

```
String result = Arrays.stream(sentence.split("\\s+"))
    .map(w -> w.toLowerCase(Locale.ROOT)).collect(Collectors.joining(" "));
```

Sample input: sentence = "JAVA API"

Sample output: "java api"

62. Reverse every word

Preserve word order while reversing each token.

```
String result = Arrays.stream(sentence.split("\\s+"))
    .map(w -> new StringBuilder(w).reverse().toString())
    .collect(Collectors.joining(" "));
```

Sample input: sentence = "Java API"

Sample output: "avaJ IPA"

63. Reverse word order

Collect the tokens in reverse encounter order.

```
String result = Arrays.stream(sentence.trim().split("\\s+"))
    .collect(Collectors.collectingAndThen(Collectors.toList(), list -> {
        Collections.reverse(list); return String.join(" ", list);
    }));
```

Sample input: sentence = "Java Stream API"

Sample output: "API Stream Java"

64. Remove duplicate words

Use distinct after normalizing case.

```
List<String> unique = Arrays.stream(sentence.split("\\s+"))
    .map(w -> w.toLowerCase(Locale.ROOT)).distinct()
    .collect(Collectors.toList());
```

Sample input: sentence = "Java java API Java"

Sample output: ["java", "api"]

65. Sort words alphabetically

Sort tokens by natural string order.

```
List<String> sorted = Arrays.stream(sentence.split("\\s+"))
    .sorted().collect(Collectors.toList());
```

Sample input: sentence = "Java API Stream"

Sample output: ["API", "Java", "Stream"]

66. Sort words by length

Order short words before longer ones.

```
List<String> sorted = Arrays.stream(sentence.split("\\s+"))
    .sorted(Comparator.comparingInt(String::length)).collect(Collectors.toList());
```

Sample input: sentence = "bb a c"

Sample output: ["a", "c", "bb"]

67. Sort words by length then alphabetically

Use a comparator chain for deterministic order.

```
List<String> sorted = Arrays.stream(sentence.split("\\s+"))
    .sorted(Comparator.comparingInt(String::length).thenComparing(Comparator.naturalOrder()))
    .collect(Collectors.toList());
```

Sample input: sentence = "bb a c"

Sample output: ["a", "c", "bb"]

68. Find words starting with a vowel

Select words with a vowel as their first letter.

```
List<String> result = Arrays.stream(sentence.split("\\s+"))
    .filter(w -> !w.isEmpty() && "aeiouAEIOU".indexOf(w.charAt(0)) >= 0)
    .collect(Collectors.toList());
```

Sample input: sentence = "apple banana orange"

Sample output: ["apple", "orange"]

69. Find words ending in a vowel

Select words with a vowel as their final letter.

```
List<String> result = Arrays.stream(sentence.split("\\s+"))
    .filter(w -> !w.isEmpty() && "aeiouAEIOU".indexOf(w.charAt(w.length() - 1)) >= 0)
    .collect(Collectors.toList());
```

Sample input: sentence = "java api code"

Sample output: ["java", "api", "code"]

70. Find palindrome words

Compare each lowercase word with its reverse.

```
List<String> palindromes = Arrays.stream(sentence.split("\\s+"))
    .filter(w -> w.equalsIgnoreCase(new StringBuilder(w).reverse().toString()))
    .collect(Collectors.toList());
```

Sample input: sentence = "level java radar"

Sample output: ["level", "radar"]

71. Keep words containing a digit

Find alphanumeric tokens.

```
List<String> result = Arrays.stream(sentence.split("\\s+"))
    .filter(w -> w.chars().anyMatch(Character::isDigit))
    .collect(Collectors.toList());
```

Sample input: sentence = "a1 java 2b"

Sample output: ["a1", "2b"]

72. Remove punctuation from sentence words

Clean each token before joining again.

```
String clean = Arrays.stream(sentence.split("\\s+"))
    .map(w -> w.replaceAll("[^A-Za-z0-9]", ""))
    .filter(w -> !w.isEmpty()).collect(Collectors.joining(" "));
```

Sample input: sentence = "Java, Stream!"

Sample output: "Java Stream"

73. Build initials from words

Take the first letter of every non-empty token.

```
String initials = Arrays.stream(sentence.trim().split("\\s+"))
    .filter(w -> !w.isEmpty()).map(w -> w.substring(0, 1).toUpperCase(Locale.ROOT))
    .collect(Collectors.joining("."));
```

Sample input: sentence = "Java Stream API"

Sample output: "J.S.A"

74. Join words with commas

Use joining as a terminal collector.

```
String csv = Arrays.stream(sentence.split("\\s+"))
    .collect(Collectors.joining(","));
```

Sample input: sentence = "Java Stream API"

Sample output: "Java,Stream,API"

75. Create a title-case sentence

Capitalize each token's first character.

```
String title = Arrays.stream(sentence.trim().split("\\s+"))
    .map(w -> w.substring(0, 1).toUpperCase(Locale.ROOT) + w.substring(1).toLowerCase(Locale.ROOT))
    .collect(Collectors.joining(" "));
```

Sample input: sentence = "java STREAM api"

Sample output: "Java Stream Api"

76. Create camelCase from words

Lowercase the first word and capitalize the rest.

```
List<String> parts = Arrays.stream(sentence.trim().split("\\s+"))
    .map(w -> w.toLowerCase(Locale.ROOT)).collect(Collectors.toList());
String camel = IntStream.range(0, parts.size()).mapToObj(i -> i == 0 ? parts.get(i)
    : parts.get(i).substring(0, 1).toUpperCase(Locale.ROOT) + parts.get(i).substring(1))
    .collect(Collectors.joining());
```

Sample input: sentence = "java stream api"

Sample output: "javaStreamApi"

77. Create snake_case from words

Normalize then join with underscores.

```
String snake = Arrays.stream(sentence.trim().split("\\s+"))
    .map(w -> w.toLowerCase(Locale.ROOT)).collect(Collectors.joining("_"));
```

Sample input: sentence = "java stream api"

Sample output: "java_stream_api"

78. Create kebab-case from words

Normalize then join with hyphens.

```
String kebab = Arrays.stream(sentence.trim().split("\\s+"))
    .map(w -> w.toLowerCase(Locale.ROOT)).collect(Collectors.joining("-"));
```

Sample input: sentence = "java stream api"

Sample output: "java-stream-api"

79. Find common words between two sentences

Put one sentence in a set and filter the other.

```
Set<String> right = Arrays.stream(second.split("\\s+"))
    .map(w -> w.toLowerCase(Locale.ROOT)).collect(Collectors.toSet());
List<String> common = Arrays.stream(first.split("\\s+"))
    .filter(w -> right.contains(w.toLowerCase(Locale.ROOT))).distinct().collect(Collectors.toList());
```

Sample input: first = "java stream api"; second = "api spring java"

Sample output: ["java", "api"]

4. Lists of strings

80. Remove duplicate strings

Keep original encounter order.

```
List<String> unique = words.stream()
    .distinct().collect(Collectors.toList());
```

Sample input: words = ["java", "api", "java"]

Sample output: ["java", "api"]

81. Remove duplicates ignoring case

Normalize first, then use distinct.

```
List<String> unique = words.stream()
    .map(s -> s.toLowerCase(Locale.ROOT)).distinct().collect(Collectors.toList());
```

Sample input: words = ["Java", "java", "API"]

Sample output: ["java", "api"]

82. Sort strings alphabetically

Sort in natural order.

```
List<String> sorted = words.stream()
    .sorted().collect(Collectors.toList());
```

Sample input: words = ["java", "API", "stream"]

Sample output: ["API", "java", "stream"]

83. Sort strings alphabetically ignoring case

Use CASE_INSENSITIVE_ORDER.

```
List<String> sorted = words.stream()
    .sorted(String.CASE_INSENSITIVE_ORDER).collect(Collectors.toList());
```

Sample input: words = ["Java", "api", "stream"]

Sample output: ["api", "Java", "stream"]

84. Sort strings in reverse alphabetical order

Reverse natural ordering.

```
List<String> sorted = words.stream()
    .sorted(Comparator.reverseOrder()).collect(Collectors.toList());
```

Sample input: words = ["java", "API", "stream"]

Sample output: ["stream", "java", "API"]

85. Sort strings by length

Keep ties in original order because stream sorting is stable.

```
List<String> sorted = words.stream()
    .sorted(Comparator.comparingInt(String::length)).collect(Collectors.toList());
```

Sample input: words = ["Java", "API", "stream"]

Sample output: ["API", "Java", "stream"]

86. Sort strings by length descending

Show longest values first.

```
List<String> sorted = words.stream()
    .sorted(Comparator.comparingInt(String::length).reversed()).collect(Collectors.toList());
```

Sample input: words = ["Java", "API", "stream"]

Sample output: ["stream", "Java", "API"]

87. Find the longest string

Find the maximum by length.

```
Optional<String> longest = words.stream()
    .max(Comparator.comparingInt(String::length));
```

Sample input: words = ["java", "api", "stream"]

Sample output: "stream"

88. Find the shortest string

Find the minimum by length.

```
Optional<String> shortest = words.stream()
    .min(Comparator.comparingInt(String::length));
```

Sample input: words = ["java", "api", "stream"]

Sample output: "api"

89. Find the first string alphabetically

Use min with natural ordering.

```
Optional<String> first = words.stream()
    .min(Comparator.naturalOrder());
```

Sample input: words = ["Java", "api", "stream"]

Sample output: "Java"

90. Find the last string alphabetically

Use max with natural ordering.

```
Optional<String> last = words.stream()
    .max(Comparator.naturalOrder());
```

Sample input: words = ["Java", "api", "stream"]

Sample output: "stream"

91. Get the top three longest strings

Sort by length descending and limit.

```
List<String> top3 = words.stream()
    .sorted(Comparator.comparingInt(String::length).reversed()).limit(3)
    .collect(Collectors.toList());
```

Sample input: words = ["a", "java", "streams", "programming"]

Sample output: ["programming", "streams", "java"]

92. Skip the first two strings

Use skip for simple paging.

```
List<String> rest = words.stream()
    .skip(2).collect(Collectors.toList());
```

Sample input: words = ["java", "api", "stream"]

Sample output: ["stream"]

93. Get a page of strings

Skip an offset, then limit page size.

```
List<String> page = words.stream()
    .skip((long) pageNumber * pageSize).limit(pageSize)
    .collect(Collectors.toList());
```

Sample input: words = ["java", "api", "stream"]; pageNumber = 1; pageSize = 1

Sample output: ["api"]

94. Count strings longer than five characters

Filter and count.

```
long count = words.stream()
    .filter(s -> s.length() > 5).count();
```

Sample input: words = ["java", "streams", "api", "program"]

Sample output: 2

95. Check if any string is empty

Use anyMatch for a short-circuiting test.

```
boolean hasEmpty = words.stream()
    .anyMatch(String::isEmpty);
```

Sample input: words = ["", "java"]

Sample output: true

96. Check if all strings are non-empty

Use allMatch for a simple validation.

```
boolean allNonEmpty = words.stream()
    .allMatch(s -> !s.isEmpty());
```

Sample input: words = ["java", "api"]

Sample output: true

97. Check if no string contains a space

Test the full list with noneMatch.

```
boolean noSpaces = words.stream()
    .noneMatch(s -> s.contains(" "));
```

Sample input: words = ["java", "api"]

Sample output: true

98. Find a string by exact value

Return Optional instead of null.

```
Optional<String> match = words.stream()
    .filter("Java"::equals).findFirst();
```

Sample input: words = ["api", "Java"]

Sample output: "Java"

99. Find a string ignoring case

Use equalsIgnoreCase in the predicate.

```
Optional<String> match = words.stream()
    .filter(s -> s.equalsIgnoreCase("java")).findFirst();
```

Sample input: words = ["api", "Java"]

Sample output: "Java"

100. Get strings with unique lengths

Keep only one representative per length.

```
List<String> uniqueLengths = words.stream()
    .collect(Collectors.toMap(String::length, s -> s, (first, ignored) -> first, LinkedHashMap::new))
    .values().stream().collect(Collectors.toList());
```

Sample input: words = ["java", "api", "cat", "dog", "stream"]

Sample output: ["java", "api", "stream"]

101. Partition strings by length

Create true/false groups for a condition.

```
Map<Boolean, List<String>> partition = words.stream()
    .collect(Collectors.partitioningBy(s -> s.length() > 5));
```

Sample input: words = ["java", "api", "stream"]

Sample output: true -> ["stream"], false -> ["java", "api"]

102. Group strings by first character

Group values by their initial letter.

```
Map<Character, List<String>> byInitial = words.stream()
    .filter(s -> !s.isEmpty()).collect(Collectors.groupingBy(s -> s.charAt(0)));
```

Sample input: words = ["Java", "API", "java"]

Sample output: map contents (entry order may vary): J -> ["Java"], A -> ["API"], j -> ["java"]

103. Group strings by length

Build a length-to-words map.

```
Map<Integer, List<String>> byLength = words.stream()
    .collect(Collectors.groupingBy(String::length));
```

Sample input: words = ["java", "api", "cat"]

Sample output: map contents (entry order may vary): 4 -> ["java"], 3 -> ["api", "cat"]

104. Join a list with brackets

Use joining prefix and suffix options.

```
String display = words.stream()
    .collect(Collectors.joining(", ", "[", "]"));
```

Sample input: words = ["java", "api"]

Sample output: "[java, api]"

105. Create an index-to-string map

Use a stream of indexes; values must have no duplicate keys.

```
Map<Integer, String> indexed = IntStream.range(0, words.size()).boxed()
    .collect(Collectors.toMap(i -> i, words::get));
```

Sample input: words = ["java", "api"]

Sample output: map contents (entry order may vary): 0 -> "java", 1 -> "api"

5. Frequency, grouping, and counting

106. Count occurrences of each word

Use groupingBy with counting.

```
Map<String, Long> counts = words.stream()
    .collect(Collectors.groupingBy(Function.identity(), Collectors.counting()));
```

Sample input: words = ["java", "api", "java"]

Sample output: map contents (entry order may vary): java -> 2, api -> 1

107. Count occurrences ignoring case

Normalize each word before grouping.

```
Map<String, Long> counts = words.stream()
    .map(s -> s.toLowerCase(Locale.ROOT))
    .collect(Collectors.groupingBy(Function.identity(), Collectors.counting()));
```

Sample input: words = ["Java", "java", "API"]

Sample output: map contents (entry order may vary): java -> 2, api -> 1

108. Count character occurrences

Create a character key for every code unit.

```
Map<Character, Long> counts = text.chars().mapToObj(c -> (char) c)
    .collect(Collectors.groupingBy(Function.identity(), Collectors.counting()));
```

Sample input: text = "programming"

Sample output: map contents (entry order may vary): p -> 1, r -> 2, o -> 1, g -> 2, a -> 1, m -> 2, i -> 1, n -> 1

109. Count character occurrences in order

Use LinkedHashMap to preserve first-seen key order.

```
Map<Character, Long> counts = text.chars().mapToObj(c -> (char) c)
    .collect(Collectors.groupingBy(Function.identity(), LinkedHashMap::new, Collectors.counting()));
```

Sample input: text = "programming"

Sample output: p -> 1, r -> 2, o -> 1, g -> 2, a -> 1, m -> 2, i -> 1, n -> 1

110. Find duplicate strings

Keep values whose frequency is greater than one.

```
Map<String, Long> counts = words.stream()
    .collect(Collectors.groupingBy(Function.identity(), Collectors.counting()));
List<String> duplicates = counts.entrySet().stream().filter(e -> e.getValue() > 1)
    .map(Map.Entry::getKey).collect(Collectors.toList());
```

Sample input: words = ["java", "api", "java"]

Sample output: ["java"] (list order may vary)

111. Find unique strings

Keep values whose frequency is exactly one.

```
Map<String, Long> counts = words.stream()
    .collect(Collectors.groupingBy(Function.identity(), Collectors.counting()));
List<String> singles = counts.entrySet().stream().filter(e -> e.getValue() == 1)
    .map(Map.Entry::getKey).collect(Collectors.toList());
```

Sample input: words = ["java", "api", "java"]

Sample output: ["api"]

112. Find the first non-repeated character

Count first, then search in original input order.

```
Map<Character, Long> counts = text.chars().mapToObj(c -> (char) c)
    .collect(Collectors.groupingBy(Function.identity(), Collectors.counting()));
Optional<Character> firstUnique = text.chars().mapToObj(c -> (char) c)
    .filter(c -> counts.get(c) == 1).findFirst();
```

Sample input: text = "swiss"

Sample output: w

113. Find the most frequent string

Maximize an entry by its count.

```
Optional<String> mostFrequent = words.stream()
    .collect(Collectors.groupingBy(Function.identity(), Collectors.counting())).entrySet().stream()
    .max(Map.Entry.comparingByValue()).map(Map.Entry::getKey);
```

Sample input: words = ["java", "api", "java"]

Sample output: "java"

114. Find the least frequent string

Minimize an entry by its count.

```
Optional<String> leastFrequent = words.stream()
    .collect(Collectors.groupingBy(Function.identity(), Collectors.counting())).entrySet().stream()
    .min(Map.Entry.comparingByValue()).map(Map.Entry::getKey);
```

Sample input: words = ["java", "java", "api"]

Sample output: "api"

115. Sort strings by frequency descending

Count, sort map entries, and extract keys.

```
List<String> ranked = words.stream()
    .collect(Collectors.groupingBy(Function.identity(), Collectors.counting()))
    .entrySet().stream()
    .sorted(Map.Entry.<String, Long>comparingByValue().reversed()).map(Map.Entry::getKey)
    .collect(Collectors.toList());
```

Sample input: words = ["java", "java", "api", "stream", "stream", "stream"]

Sample output: ["stream", "java", "api"]

116. Sort strings by frequency then name

Make equal-count ordering predictable.

```
List<String> ranked = words.stream()
    .collect(Collectors.groupingBy(Function.identity(), Collectors.counting()))
    .entrySet().stream().sorted(Map.Entry.<String, Long>comparingByValue().reversed()
    .thenComparing(Map.Entry.comparingByKey())).map(Map.Entry::getKey).collect(Collectors.toList());
```

Sample input: words = ["java", "java", "api", "stream", "stream", "stream"]

Sample output: ["stream", "java", "api"]

117. Get the frequency map sorted by key

Use TreeMap as the grouping map supplier.

```
Map<String, Long> sortedCounts = words.stream()
    .collect(Collectors.groupingBy(Function.identity(), TreeMap::new, Collectors.counting()));
```

Sample input: words = ["Java", "api"]

Sample output: Java -> 1, api -> 1

118. Count strings by length

Aggregate only the count for each length.

```
Map<Integer, Long> byLength = words.stream()
    .collect(Collectors.groupingBy(String::length, Collectors.counting()));
```

Sample input: words = ["java", "api", "cat"]

Sample output: map contents (entry order may vary): 4 -> 1, 3 -> 2

119. Count strings by first letter

Aggregate the count for each initial.

```
Map<Character, Long> byInitial = words.stream().filter(s -> !s.isEmpty())
    .collect(Collectors.groupingBy(s -> s.charAt(0), Collectors.counting()));
```

Sample input: words = ["Java", "API", "java"]

Sample output: map contents (entry order may vary): J -> 1, A -> 1, j -> 1

120. Group anagrams

Use sorted lowercase letters as a canonical key.

```
Map<String, List<String>> anagrams = words.stream().collect(Collectors.groupingBy(w ->
    w.toLowerCase(Locale.ROOT).chars().sorted().mapToObj(c -> String.valueOf((char) c))
    ).collect(Collectors.joining())));
```

Sample input: words = ["eat", "tea", "tan"]

Sample output: map contents (entry order may vary): aet -> ["eat", "tea"], ant -> ["tan"]

121. Find anagram groups only

Keep canonical groups that have at least two words.

```
Map<String, List<String>> groups = words.stream().collect(Collectors.groupingBy(w ->
    w.toLowerCase(Locale.ROOT).chars().sorted().mapToObj(c -> String.valueOf((char) c))
    .collect(Collectors.joining())));
List<List<String>> anagramGroups = groups.values().stream()
    .filter(g -> g.size() > 1).collect(Collectors.toList());
```

Sample input: words = ["eat", "tea", "tan"]

Sample output: [["eat", "tea"]]

122. Count vowels by vowel character

Group vowels rather than only counting their total.

```
Map<Character, Long> vowelCounts = text.toLowerCase(Locale.ROOT).chars()
    .filter(c -> "aeiou".indexOf(c) >= 0).mapToObj(c -> (char) c)
    .collect(Collectors.groupingBy(Function.identity(), Collectors.counting()));
```

Sample input: text = "Education"

Sample output: map contents (entry order may vary): a -> 1, e -> 1, i -> 1, o -> 1, u -> 1

123. Count first letters of sentence words

Tokenize then aggregate by initial.

```
Map<Character, Long> initials = Arrays.stream(sentence.trim().split("\\s+"))
    .filter(w -> !w.isEmpty())
    .collect(Collectors.groupingBy(w -> Character.toLowerCase(w.charAt(0)), Collectors.counting()));
```

Sample input: sentence = "Java API apple"

Sample output: map contents (entry order may vary): j -> 1, a -> 2

124. Group words by their last character

Bucket words based on their final character.

```
Map<Character, List<String>> byLast = words.stream().filter(s -> !s.isEmpty())
    .collect(Collectors.groupingBy(s -> s.charAt(s.length() - 1)));
```

Sample input: words = ["java", "api", "code"]

Sample output: map contents (entry order may vary): a -> ["java"], i -> ["api"], e -> ["code"]

125. Group strings by uppercase first letter

Normalize group keys for case-insensitive grouping.

```
Map<Character, List<String>> byInitial = words.stream().filter(s -> !s.isEmpty())
    .collect(Collectors.groupingBy(s -> Character.toUpperCase(s.charAt(0))));
```

Sample input: words = ["Java", "API", "apple"]

Sample output: map contents (entry order may vary): J -> ["Java"], A -> ["API", "apple"]

126. Count words after punctuation cleanup

Normalize words before building the frequency map.

```
Map<String, Long> counts = Arrays.stream(sentence.split("\\s+"))
    .map(w -> w.replaceAll("[^A-Za-z0-9]", "").toLowerCase(Locale.ROOT)).filter(w -> !w.isEmpty())
    .collect(Collectors.groupingBy(Function.identity(), Collectors.counting()));
```

Sample input: sentence = "Java, java API"

Sample output: map contents (entry order may vary): java -> 2, api -> 1

127. Find the highest-frequency character

Count and then maximize by count.

```
Optional<Character> mostFrequent = text.chars().mapToObj(c -> (char) c)
    .collect(Collectors.groupingBy(Function.identity(), Collectors.counting()))
    .entrySet().stream()
    .max(Map.Entry.comparingByValue()).map(Map.Entry::getKey);
```

Sample input: text = "AAABBC"

Sample output: "A"

128. Find characters occurring more than twice

Filter a character count map.

```
List<Character> frequent = text.chars().mapToObj(c -> (char) c)
    .collect(Collectors.groupingBy(Function.identity(), Collectors.counting()))
    .entrySet().stream()
    .filter(e -> e.getValue() > 2).map(Map.Entry::getKey).collect(Collectors.toList());
```

Sample input: text = "aabbbc"

Sample output: [b]

129. Build a word-length histogram

Join key/value entries into a printable summary.

```
String histogram = words.stream()
    .collect(Collectors.groupingBy(String::length, TreeMap::new, Collectors.counting()))
    .entrySet().stream().map(e -> e.getKey() + ":" + e.getValue())
    .collect(Collectors.joining(", "));
```

Sample input: words = ["java", "api", "cat"]

Sample output: "3:2, 4:1"

130. Group words by their length parity

Use a descriptive string key.

```
Map<String, List<String>> parity = words.stream()
    .collect(Collectors.groupingBy(w -> w.length() % 2 == 0 ? "even" : "odd"));
```

Sample input: words = ["java", "api", "cat"]

Sample output: map contents (entry order may vary): even -> ["java"], odd -> ["api", "cat"]

131. Map each initial to joined words

Use mapping downstream before joining.

```
Map<Character, String> joined = words.stream().filter(s -> !s.isEmpty())
    .collect(Collectors.groupingBy(s -> s.charAt(0),
        Collectors.mapping(Function.identity(), Collectors.joining(", "))));
```

Sample input: words = ["Java", "API", "apple"]

Sample output: map contents (entry order may vary): J -> "Java", A -> "API", a -> "apple"

132. Group words by length as comma-separated text

Use a downstream joining collector.

```
Map<Integer, String> grouped = words.stream()
    .collect(Collectors.groupingBy(String::length, Collectors.joining(", ")));
```

Sample input: words = ["java", "api", "cat"]

Sample output: map contents (entry order may vary): 4 -> "java", 3 -> "api, cat"

133. Count anagram signatures

Find how many words belong to each canonical spelling.

```
Map<String, Long> signatures = words.stream().map(w -> w.toLowerCase(Locale.ROOT).chars().sorted()
    .mapToObj(c -> String.valueOf((char) c)).collect(Collectors.joining()))
    .collect(Collectors.groupingBy(Function.identity(), Collectors.counting()));
```

Sample input: words = ["eat", "tea", "tan"]

Sample output: map contents (entry order may vary): aet -> 2, ant -> 1

134. Find the longest word in each initial group

Compose groupingBy with a maxBy downstream collector.

```
Map<Character, Optional<String>> longestByInitial = words.stream().filter(s -> !s.isEmpty())
    .collect(Collectors.groupingBy(s -> s.charAt(0),
        Collectors.maxBy(Comparator.comparingInt(String::length))));
```

Sample input: words = ["Java", "API", "apple"]

Sample output: map contents (entry order may vary): J -> "Java", A -> "API", a -> "apple"

135. Find average string length

Map text lengths to an IntStream average.

```
OptionalDouble average = words.stream()
    .mapToInt(String::length).average();
```

Sample input: words = ["java", "api", "stream"]

Sample output: 4.333333333333333

6. Matching, reduction, and Optional

136. Check whether a list contains a palindrome

Return true when any item reads the same backwards.

```
boolean hasPalindrome = words.stream()
    .anyMatch(w -> w.equalsIgnoreCase(new StringBuilder(w).reverse().toString()));
```

Sample input: words = ["level", "java"]

Sample output: true

137. Check whether all strings are palindromes

Require a non-empty list explicitly if that is your rule.

```
boolean allPalindromes = !words.isEmpty() && words.stream()
    .allMatch(w -> w.equalsIgnoreCase(new StringBuilder(w).reverse().toString()));
```

Sample input: words = ["level", "radar"]

Sample output: true

138. Check whether no string contains digits

Use noneMatch for a fast boolean test.

```
boolean noDigits = words.stream()
    .noneMatch(w -> w.chars().anyMatch(Character::isDigit));
```

Sample input: words = ["java", "api"]

Sample output: true

139. Check whether any string begins with uppercase

Inspect first character after excluding empty values.

```
boolean anyCapitalized = words.stream()
    .anyMatch(w -> !w.isEmpty() && Character.isUpperCase(w.charAt(0)));
```

Sample input: words = ["java", "API"]

Sample output: true

140. Check a string is a palindrome ignoring punctuation

Normalize before comparing with its reverse.

```
String clean = text.replaceAll("[^A-Za-z0-9]", "").toLowerCase(Locale.ROOT);
boolean palindrome = clean.chars().mapToObj(c -> String.valueOf((char) c))
    .collect(Collectors.joining()).equals(new StringBuilder(clean).reverse().toString());
```

Sample input: text = "A man, a plan, a canal: Panama"

Sample output: true

141. Check two strings are anagrams

Compare sorted, lowercase character streams.

```
String a = first.replaceAll("\\s+", "").toLowerCase(Locale.ROOT);
String b = second.replaceAll("\\s+", "").toLowerCase(Locale.ROOT);
boolean anagram = a.chars().sorted().boxed().collect(Collectors.toList())
    .equals(b.chars().sorted().boxed().collect(Collectors.toList()));
```

Sample input: first = "listen"; second = "silent"

Sample output: true

142. Concatenate all strings

Reduce with `String::concat`; joining is often clearer.

```
String combined = words.stream()
    .reduce("", String::concat);
```

Sample input: words = ["java", "api"]

Sample output: "javaapi"

143. Join all strings with a delimiter

Use joining for a delimited reduction.

```
String combined = words.stream()
    .collect(Collectors.joining(" | "));
```

Sample input: words = ["java", "api"]

Sample output: "java | api"

144. Find the first string longer than ten

Use `findFirst` to preserve list order.

```
Optional<String> firstLong = words.stream()
    .filter(s -> s.length() > 10).findFirst();
```

Sample input: words = ["tiny", "extraordinary", "longvalue"]

Sample output: "extraordinary"

145. Find any string containing a keyword

`findAny` may be useful with parallel streams.

```
Optional<String> match = words.stream()
    .filter(s -> s.contains(keyword)).findAny();
```

Sample input: words = ["java", "stream", "api"]; keyword = "str"

Sample output: "stream"

146. Find the second longest distinct string

Sort descending, skip first, and find first.

```
Optional<String> secondLongest = words.stream().distinct()
    .sorted(Comparator.comparingInt(String::length).reversed()).skip(1).findFirst();
```

Sample input: words = ["java", "stream", "api", "stream"]

Sample output: "java"

147. Get the first three-character string or default

Finish Optional with orElse.

```
String value = words.stream()
    .filter(s -> s.length() == 3).findFirst().orElse("N/A");
```

Sample input: words = ["a", "bb"]

Sample output: "N/A"

148. Get longest string length with a default

Map Optional<String> to Optional<Integer>.

```
int length = words.stream().max(Comparator.comparingInt(String::length))
    .map(String::length).orElse(0);
```

Sample input: words = []

Sample output: 0

149. Reduce to the longest string

Choose the longer of each pair.

```
Optional<String> longest = words.stream()
    .reduce((left, right) -> left.length() >= right.length() ? left : right);
```

Sample input: words = ["java", "streams", "api"]

Sample output: "streams"

150. Reduce to the shortest string

Choose the shorter of each pair.

```
Optional<String> shortest = words.stream()
    .reduce((left, right) -> left.length() <= right.length() ? left : right);
```

Sample input: words = ["java", "streams", "api"]

Sample output: "api"

151. Calculate total character count

Map each string to its length then sum.

```
int total = words.stream()
    .mapToInt(String::length).sum();
```

Sample input: words = ["java", "streams", "api"]

Sample output: 13

152. Calculate average word length

Get an OptionalDouble for an empty list.

```
OptionalDouble average = words.stream()
    .mapToInt(String::length).average();
```

Sample input: words = ["java", "streams", "api"]

Sample output: 4.333333333333333

153. Find a string with maximum vowel count

Use a computed comparator key.

```
Optional<String> mostVowels = words.stream().max(Comparator.comparingLong(w ->
    w.toLowerCase(Locale.ROOT).chars().filter(c -> "aeiou".indexOf(c) >= 0).count()));
```

Sample input: words = ["aeiou", "java"]

Sample output: "aeiou"

154. Find a string with minimum digit count

Use a computed comparator key.

```
Optional<String> fewestDigits = words.stream().min(Comparator.comparingLong(w ->
    w.chars().filter(Character::isDigit).count()));
```

Sample input: words = ["a1", "b22", "c"]

Sample output: "c"

155. Find the lexicographically first non-blank string

Filter then use sorted/findFirst.

```
Optional<String> first = words.stream().filter(s -> !s.trim().isEmpty())
    .sorted().findFirst();
```

Sample input: words = [" ", "java", "API"]

Sample output: "API"

7. Collectors and map recipes

156. Collect unique strings into a set

Use toSet when set ordering is unimportant.

```
Set<String> unique = words.stream()
    .collect(Collectors.toSet());
```

Sample input: words = ["java", "api", "java"]

Sample output: set contents (order may vary): "java", "api"

157. Collect unique strings into an ordered set

Use LinkedHashSet to preserve encounter order.

```
Set<String> unique = words.stream()
    .collect(Collectors.toCollection(LinkedHashSet::new));
```

Sample input: words = ["java", "api", "java"]

Sample output: ["java", "api"]

158. Collect strings into a sorted set

Use TreeSet for natural ordering.

```
Set<String> sorted = words.stream()
    .collect(Collectors.toCollection(TreeSet::new));
```

Sample input: words = ["java", "api", "java"]

Sample output: set contents: "api", "java"

159. Create a length-to-string map

Supply a merge rule for equal lengths.

```
Map<Integer, String> byLength = words.stream()
    .collect(Collectors.toMap(String::length, Function.identity(), (first, ignored) -> first));
```

Sample input: words = ["java", "api", "go"]

Sample output: map contents (entry order may vary): 4 -> "java", 3 -> "api", 2 -> "go"

160. Create a string-to-length map

Text values become unique map keys.

```
Map<String, Integer> lengths = words.stream()
    .collect(Collectors.toMap(Function.identity(), String::length));
```

Sample input: words = ["java", "api"]

Sample output: map contents (entry order may vary): java -> 4, api -> 3

161. Create a lowercase-to-original map

Keep the first casing when keys collide.

```
Map<String, String> originals = words.stream().collect(Collectors.toMap(
    s -> s.toLowerCase(Locale.ROOT), Function.identity(), (first, ignored) -> first));
```

Sample input: words = ["Java", "java", "API"]

Sample output: map contents (entry order may vary): java -> "Java", api -> "API"

162. Create an initial-to-count map

Use groupingBy and counting for a compact collector.

```
Map<Character, Long> counts = words.stream().filter(s -> !s.isEmpty())
    .collect(Collectors.groupingBy(s -> s.charAt(0), Collectors.counting()));
```

Sample input: words = ["Java", "API", "java"]

Sample output: map contents (entry order may vary): J -> 1, A -> 1, j -> 1

163. Join strings with prefix and suffix

Produce a readable single string.

```
String result = words.stream()
    .collect(Collectors.joining(", ", "Words: ", "."));
```

Sample input: words = ["java", "api"]

Sample output: "Words: java, api."

164. Join only nonblank strings

Filter before joining.

```
String result = words.stream().filter(s -> !s.trim().isEmpty())
    .collect(Collectors.joining(", "));
```

Sample input: words = [" ", "java", ""]

Sample output: "java"

165. Collect string lengths

Map to Integer values.

```
List<Integer> lengths = words.stream()
    .map(String::length).collect(Collectors.toList());
```

Sample input: words = ["java", "api"]

Sample output: [4, 3]

166. Collect first characters

Exclude empty strings before charAt.

```
List<Character> initials = words.stream().filter(s -> !s.isEmpty())
    .map(s -> s.charAt(0)).collect(Collectors.toList());
```

Sample input: words = ["Java", "API"]

Sample output: [J, A]

167. Flatten a list of comma-separated strings

Split each item then flatten the nested streams.

```
List<String> tokens = lines.stream()
    .flatMap(line -> Arrays.stream(line.split(", ")))
    .map(String::trim)
    .collect(Collectors.toList());
```

Sample input: lines = ["java, api", "stream"]

Sample output: ["java", "api", "stream"]

168. Flatten words from multiple sentences

Split every sentence and flatten all tokens.

```
List<String> tokens = sentences.stream()
    .flatMap(s -> Arrays.stream(s.trim().split("\\s+")))
    .filter(s -> !s.isEmpty()).collect(Collectors.toList());
```

Sample input: sentences = ["Java is powerful", "Stream API is useful"]

Sample output: ["Java", "is", "powerful", "Stream", "API", "is", "useful"]

169. Collect all distinct characters from a list

Flatten character streams then use distinct.

```
Set<Character> characters = words.stream()
    .flatMapToInt(String::chars).mapToObj(c -> (char) c)
    .collect(Collectors.toCollection(LinkedHashSet::new));
```

Sample input: words = ["java", "api"]

Sample output: [j, a, v, p, i]

170. Build a character set from a string

Use a collector to preserve encounter order.

```
Set<Character> characters = text.chars().mapToObj(c -> (char) c)
    .collect(Collectors.toCollection(LinkedHashSet::new));
```

Sample input: text = "programming"

Sample output: [p, r, o, g, a, m, i, n]

171. Collect words into an ArrayDeque

Choose the desired concrete collection type.

```
Deque<String> queue = words.stream()
    .collect(Collectors.toCollection(ArrayDeque::new));
```

Sample input: words = ["java", "api"]

Sample output: ["java", "api"]

172. Create immutable-style copied list

Collect a mutable ArrayList; wrap if an immutable view is needed.

```
List<String> copy = words.stream()
    .collect(Collectors.toCollection(ArrayList::new));
```

Sample input: words = ["java", "api"]

Sample output: ["java", "api"]

173. Build a sorted frequency map

Count words in a TreeMap.

```
Map<String, Long> frequency = words.stream()
    .collect(Collectors.groupingBy(Function.identity(), TreeMap::new, Collectors.counting()));
```

Sample input: words = ["java", "api", "java"]

Sample output: api -> 1, java -> 2

174. Partition names by upper/lowercase start

Partition values using a boolean predicate.

```
Map<Boolean, List<String>> grouped = names.stream().filter(s -> !s.isEmpty())
    .collect(Collectors.partitioningBy(s -> Character.isUpperCase(s.charAt(0))));
```

Sample input: names = ["Anna", "bob", "Ava"]

Sample output: true -> ["Anna", "Ava"], false -> ["bob"]

175. Summarize string lengths

Get count, sum, min, max, and average together.

```
IntSummaryStatistics stats = words.stream()
    .collect(Collectors.summarizingInt(String::length));
```

Sample input: words = ["java", "api", "stream"]

Sample output: count=3, sum=13, min=3, max=6, average=4.333333333333333

8. Transformations and practical recipes

176. Create URL slugs

Lowercase, trim, normalize spaces, and remove unsupported characters.

```
List<String> slugs = titles.stream().map(s -> s.toLowerCase(Locale.ROOT).trim()
    .replaceAll("\\s+", "-").replaceAll("[^a-z0-9-]", ""))
    .collect(Collectors.toList());
```

Sample input: titles = ["Java Stream API"]

Sample output: ["java-stream-api"]

177. Mask all but the last four characters

Keep short strings unchanged.

```
List<String> masked = values.stream().map(s -> s.length() <= 4 ? s
    : IntStream.range(0, s.length() - 4).mapToObj(i -> "**")
    .collect(Collectors.joining()) + s.substring(s.length() - 4))
    .collect(Collectors.toList());
```

Sample input: values = ["123456789"]

Sample output: ["*****6789"]

178. Mask email usernames

Retain first user character and the domain.

```
List<String> masked = emails.stream().filter(e -> e.contains("@"))
    .map(e -> e.substring(0, 1) + "****" + e.substring(e.indexOf('@') + 1))
    .collect(Collectors.toList());
```

Sample input: emails = ["ana@example.com"]

Sample output: ["a***@example.com"]

179. Extract domains from emails

Keep text after @ from simple email-shaped strings.

```
List<String> domains = emails.stream().filter(e -> e.contains("@"))
    .map(e -> e.substring(e.indexOf('@') + 1).toLowerCase(Locale.ROOT))
    .collect(Collectors.toList());
```

Sample input: emails = ["ana@example.com", "sam@test.org"]

Sample output: ["example.com", "test.org"]

180. Find duplicate email domains

Extract domains then keep counts above one.

```
Map<String, Long> counts = emails.stream().filter(e -> e.contains("@"))
    .map(e -> e.substring(e.indexOf('@') + 1).toLowerCase(Locale.ROOT))
    .collect(Collectors.groupingBy(Function.identity(), Collectors.counting()));
```

Sample input: emails = ["ana@example.com", "bob@example.com", "sam@test.org"]

Sample output: map contents (entry order may vary): example.com -> 2, test.org -> 1

181. Extract hashtags from a post

Split on whitespace and retain hash-prefixed tokens.

```
List<String> tags = Arrays.stream(post.split("\\s+"))
    .filter(s -> s.startsWith("#"))
    .map(s -> s.replaceAll("[^A-Za-z0-9_]", ""))
    .collect(Collectors.toList());
```

Sample input: post = "Learning #Java with @ana"

Sample output: ["#Java"]

182. Extract @mentions from a post

Split on whitespace and retain @-prefixed tokens.

```
List<String> mentions = Arrays.stream(post.split("\\s+"))
    .filter(s -> s.startsWith("@"))
    .map(s -> s.replaceAll("[^A-Za-z0-9_]", ""))
    .collect(Collectors.toList());
```

Sample input: post = "Learning #Java with @ana"

Sample output: ["@ana"]

183. Convert CSV values to integers

Validate signed integer text before parsing.

```
List<Integer> numbers = Arrays.stream(csv.split(","))
    .map(String::trim)
    .filter(s -> s.matches("[+-]?\\d+"))
    .map(Integer::valueOf)
    .collect(Collectors.toList());
```

Sample input: csv = "10, -2, x, 30"

Sample output: [10, -2, 30]

184. Extract hexadecimal colors

Filter tokens that match RGB hex notation.

```
List<String> colors = values.stream()
    .filter(s -> s.matches("#[0-9A-Fa-f]{6}"))
    .collect(Collectors.toList());
```

Sample input: values = ["#FF0000", "#xyz", "00FF00"]

Sample output: ["#FF0000"]

185. Extract valid Java-like identifiers

Apply a simple identifier rule.

```
List<String> ids = values.stream()
    .filter(s -> s.matches("[A-Za-z_][A-Za-z0-9_]*"))
    .collect(Collectors.toList());
```

Sample input: values = ["java_8", "8java", "\$id"]

Sample output: ["java_8", "\$id"]

186. Convert a property list to a map

Split on the first equals sign and exclude malformed pairs.

```
Map<String, String> properties = lines.stream()
    .filter(s -> s.contains("="))
    .map(s -> s.split("=", 2))
    .collect(Collectors.toMap(a -> a[0].trim(), a -> a[1].trim(), (a, b) -> b));
```

Sample input: lines = ["host=localhost", "port=8080"]

Sample output: map contents (entry order may vary): host -> localhost, port -> 8080

187. Extract file extensions

Find the text after the final dot.

```
List<String> extensions = files.stream().filter(f -> f.lastIndexOf('.') > 0)
    .map(f -> f.substring(f.lastIndexOf('.') + 1).toLowerCase(Locale.ROOT))
    .collect(Collectors.toList());
```

Sample input: files = ["app.java", "readme", "notes.TXT"]

Sample output: ["java", "txt"]

188. Group file names by extension

Use an empty key for filenames with no extension.

```
Map<String, List<String>> byExtension = files.stream().collect(Collectors.groupingBy(f -> {
    int dot = f.lastIndexOf('.'); return dot > 0 ? f.substring(dot + 1).toLowerCase(Locale.ROOT) : "";
}));
```

Sample input: files = ["app.java", "notes.txt", "readme"]

Sample output: map contents (entry order may vary): java -> ["app.java"], txt -> ["notes.txt"], "" -> ["readme"]

189. Extract the base name of a path

Use the last slash or backslash position.

```
List<String> baseNames = paths.stream()
    .map(p -> p.substring(Math.max(p.lastIndexOf('/'), p.lastIndexOf('\\')) + 1))
    .collect(Collectors.toList());
```

Sample input: paths = ["src/App.java", "readme.md"]

Sample output: ["App.java", "readme.md"]

190. Find strings with balanced-looking parentheses

Compare opening and closing parenthesis totals; this is not full nesting validation.

```
List<String> candidates = expressions.stream().filter(s ->
    s.chars().filter(c -> c == '(').count() == s.chars().filter(c -> c == ')').count())
    .collect(Collectors.toList());
```

Sample input: expressions = ["(a+b)", "(a+b", "x"]

Sample output: ["(a+b)", "x"]

191. Generate all prefixes of a string

Stream index lengths from 1 through length.

```
List<String> prefixes = IntStream.rangeClosed(1, text.length())
    .mapToObj(i -> text.substring(0, i)).collect(Collectors.toList());
```

Sample input: text = "java"

Sample output: ["j", "ja", "jav", "java"]

192. Generate all suffixes of a string

Stream start indexes from 0 through length-1.

```
List<String> suffixes = IntStream.range(0, text.length())
    .mapToObj(text::substring).collect(Collectors.toList());
```

Sample input: text = "java"

Sample output: ["java", "ava", "va", "a"]

193. Generate all non-empty substrings

Use nested index streams; complexity is $O(n^2)$.

```
List<String> substrings = IntStream.range(0, text.length()).boxed()
    .flatMap(start -> IntStream.rangeClosed(start + 1, text.length())
        .mapToObj(end -> text.substring(start, end)))
    .collect(Collectors.toList());
```

Sample input: text = "abc"

Sample output: ["a", "ab", "abc", "b", "bc", "c"]

194. Find repeated words in a sentence

Clean and count tokens, then retain repeated keys.

```
Map<String, Long> counts = Arrays.stream(sentence.toLowerCase(Locale.ROOT).split("\\s+"))
    .map(w -> w.replaceAll("[^a-z0-9]", "")).filter(w -> !w.isEmpty())
    .collect(Collectors.groupingBy(Function.identity(), Collectors.counting()));
List<String> repeated = counts.entrySet().stream().filter(e -> e.getValue() > 1)
    .map(Map.Entry::getKey).collect(Collectors.toList());
```

Sample input: sentence = "Java java API"

Sample output: ["java"]

195. Replace selected words from a dictionary

Map each token through a replacement map with a fallback.

```
String result = Arrays.stream(sentence.split("\\s+"))
    .map(w -> replacements.getOrDefault(w, w))
    .collect(Collectors.joining(" "));
```

Sample input: sentence = "java is great"; replacements = {java -> Java}

Sample output: "Java is great"

196. Censor forbidden words

Replace case-insensitive exact token matches.

```
Set<String> forbidden = new HashSet<>(Arrays.asList("bad", "secret"));
String result = Arrays.stream(sentence.split("\\s+"))
    .map(w -> forbidden.contains(w.toLowerCase(Locale.ROOT)) ? "****" : w).collect(Collectors.joining(" "));
```

Sample input: sentence = "java is secret"

Sample output: "java is ****"

197. Collapse duplicate adjacent words

Use a small stateful collector sequentially; do not use it on parallel streams.

```
List<String> compact = words.stream().collect(ArrayList::new,
    (list, word) -> { if (list.isEmpty() || !list.get(list.size() - 1).equals(word)) list.add(word); },
    ArrayList::addAll);
```

Sample input: words = ["a", "a", "b", "b", "a"]

Sample output: ["a", "b", "a"]

198. Find the common prefix of all strings

Reduce pairwise with a prefix helper method.

```
Optional<String> prefix = words.stream()
    .reduce((a, b) -> IntStream.range(0, Math.min(a.length(), b.length()))
        .filter(i -> a.charAt(i) != b.charAt(i)).findFirst()
        .map(i -> a.substring(0, i)).orElse(a.length() <= b.length() ? a : b));
```

Sample input: words = ["flower", "flow", "flight"]

Sample output: "fl"

199. Detect whether a string has only unique characters

Compare the distinct character count with length.

```
boolean uniqueChars = text.chars()
    .distinct().count() == text.length();
```

Sample input: text = "abc"

Sample output: true

200. Check whether two strings have a common character

Build a character set from one string, then test the other.

```
Set<Integer> chars = first.chars().boxed().collect(Collectors.toSet());
boolean overlap = second.chars().anyMatch(chars::contains);
```

Sample input: first = "hello"; second = "world"

Sample output: true

9. Interview checklist gap coverage

201. Count special characters

Count characters that are neither letters, digits, nor whitespace.

```
long special = text.chars()
    .filter(c -> !Character.isLetterOrDigit(c) && !Character.isWhitespace(c))
    .count();
```

Sample input: text = "A! B@#"

Sample output: 3

202. Get the character code of every character

Return the integer code value for each UTF-16 character.

```
List<Integer> codes = text.chars()
    .boxed().collect(Collectors.toList());
```

Sample input: text = "AB"

Sample output: [65, 66]

203. Remove all whitespace

Remove spaces, tabs, and newlines instead of merely normalizing them.

```
String compact = text.chars()
    .filter(c -> !Character.isWhitespace(c))
    .mapToObj(c -> String.valueOf((char) c)).collect(Collectors.joining());
```

Sample input: text = "a b\tc"

Sample output: "abc"

204. List all duplicate characters

Return every character whose count is greater than one in first-seen order.

```
Map<Character, Long> counts = text.chars().mapToObj(c -> (char) c)
    .collect(Collectors.groupingBy(Function.identity(), LinkedHashMap::new, Collectors.counting()));
List<Character> duplicates = counts.entrySet().stream().filter(e -> e.getValue() > 1)
    .map(Map.Entry::getKey).collect(Collectors.toList());
```

Sample input: text = "AAABBCCDDD"

Sample output: [A, B, C, D]

205. List all unique characters

Return every character whose count is exactly one in first-seen order.

```
Map<Character, Long> counts = text.chars().mapToObj(c -> (char) c)
    .collect(Collectors.groupingBy(Function.identity(), LinkedHashMap::new, Collectors.counting()));
List<Character> unique = counts.entrySet().stream().filter(e -> e.getValue() == 1)
    .map(Map.Entry::getKey).collect(Collectors.toList());
```

Sample input: text = "AAABBCCD"

Sample output: [D]

206. Find the least frequent character

Minimize the character-frequency entries.

```
Optional<Character> leastFrequent = text.chars().mapToObj(c -> (char) c)
    .collect(Collectors.groupingBy(Function.identity(), Collectors.counting())).entrySet().stream()
    .min(Map.Entry.comparingByValue()).map(Map.Entry::getKey);
```

Sample input: text = "AAABBC"

Sample output: "C"

207. Find the second most frequent character

Sort frequency entries descending and skip the highest entry.

```
Optional<Character> secondMost = text.chars().mapToObj(c -> (char) c)
    .collect(Collectors.groupingBy(Function.identity(), Collectors.counting())).entrySet().stream()
    .sorted(Map.Entry.<Character, Long>comparingByValue().reversed()).skip(1)
    .map(Map.Entry::getKey).findFirst();
```

Sample input: text = "AAABBC"

Sample output: "B"

208. Find the second least frequent character

Sort frequency entries ascending and skip the lowest entry.

```
Optional<Character> secondLeast = text.chars().mapToObj(c -> (char) c)
    .collect(Collectors.groupingBy(Function.identity(), Collectors.counting())).entrySet().stream()
    .sorted(Map.Entry.comparingByValue()).skip(1).map(Map.Entry::getKey).findFirst();
```

Sample input: text = "AAABBC"

Sample output: "B"

209. Sort characters by frequency

Rank characters from most to least frequent.

```
List<Character> ranked = text.chars().mapToObj(c -> (char) c)
    .collect(Collectors.groupingBy(Function.identity(), Collectors.counting())).entrySet().stream()
    .sorted(Map.Entry.<Character, Long>comparingByValue().reversed()).map(Map.Entry::getKey)
    .collect(Collectors.toList());
```

Sample input: text = "AAABBC"

Sample output: [A, B, C]

210. Count character frequency ignoring case

Normalize case before creating the frequency map.

```
Map<Character, Long> counts = text.toLowerCase(Locale.ROOT).chars().mapToObj(c -> (char) c)
    .collect(Collectors.groupingBy(Function.identity(), LinkedHashMap::new, Collectors.counting()));
```

Sample input: text = "aAab"

Sample output: a -> 3, b -> 1

211. Count character frequency ignoring whitespace

Exclude whitespace before grouping character counts.

```
Map<Character, Long> counts = text.chars().filter(c -> !Character.isWhitespace(c))
    .mapToObj(c -> (char) c)
    .collect(Collectors.groupingBy(Function.identity(), LinkedHashMap::new, Collectors.counting()));
```

Sample input: text = "A A B"

Sample output: A -> 2, B -> 1

212. Find the first repeated word

Count normalized words, then search the original word order.

```
List<String> tokens = Arrays.stream(sentence.toLowerCase(Locale.ROOT).split("\\s+"))
    .collect(Collectors.toList());
Map<String, Long> counts = tokens.stream()
    .collect(Collectors.groupingBy(Function.identity(), Collectors.counting()));
Optional<String> firstRepeated = tokens.stream().filter(w -> counts.get(w) > 1).findFirst();
```

Sample input: sentence = "java is java API is"

Sample output: "java"

213. Find the first non-repeated word

Count normalized words, then search the original word order.

```
List<String> tokens = Arrays.stream(sentence.toLowerCase(Locale.ROOT).split("\\s+"))
    .collect(Collectors.toList());
Map<String, Long> counts = tokens.stream()
    .collect(Collectors.groupingBy(Function.identity(), Collectors.counting()));
Optional<String> firstUnique = tokens.stream().filter(w -> counts.get(w) == 1).findFirst();
```

Sample input: sentence = "java is java API is"

Sample output: "api"

214. Find the second shortest distinct word

Sort distinct words by length ascending and skip one.

```
Optional<String> secondShortest = Arrays.stream(sentence.split("\\s+"))
    .distinct().sorted(Comparator.comparingInt(String::length)).skip(1).findFirst();
```

Sample input: sentence = "a bb ccc dddd"

Sample output: "bb"

215. Find the third longest distinct word

Sort distinct words by length descending and skip two.

```
Optional<String> thirdLongest = Arrays.stream(sentence.split("\\s+"))
    .distinct().sorted(Comparator.comparingInt(String::length).reversed()).skip(2).findFirst();
```

Sample input: sentence = "a bb ccc dddd eeeee"

Sample output: "ccc"

216. Find the third shortest distinct word

Sort distinct words by length ascending and skip two.

```
Optional<String> thirdShortest = Arrays.stream(sentence.split("\\s+"))
    .distinct().sorted(Comparator.comparingInt(String::length)).skip(2).findFirst();
```

Sample input: sentence = "a bb ccc dddd eeeee"

Sample output: "ccc"

217. Get the top three most frequent words

Count words, rank by frequency, and retain three keys.

```
List<String> top3 = Arrays.stream(sentence.toLowerCase(Locale.ROOT).split("\\s+"))
    .collect(Collectors.groupingBy(Function.identity(), Collectors.counting()))
    .entrySet().stream()
    .sorted(Map.Entry.<String, Long>comparingByValue().reversed()).limit(3)
    .map(Map.Entry::getKey).collect(Collectors.toList());
```

Sample input: sentence = "java java java api api stream"

Sample output: ["java", "api", "stream"]

218. Get the top N most frequent words

Use a variable limit for a reusable ranking recipe.

```
List<String> topN = Arrays.stream(sentence.toLowerCase(Locale.ROOT).split("\\s+"))
    .collect(Collectors.groupingBy(Function.identity(), Collectors.counting()))
    .entrySet().stream()
    .sorted(Map.Entry.<String, Long>comparingByValue().reversed()).limit(n)
    .map(Map.Entry::getKey).collect(Collectors.toList());
```

Sample input: sentence = "java java java api api stream"; n = 2

Sample output: ["java", "api"]

219. Count palindrome words

Filter the word list with a palindrome predicate, then count.

```
long palindromeCount = words.stream()
    .filter(w -> w.equalsIgnoreCase(new StringBuilder(w).reverse().toString()))
    .count();
```

Sample input: words = ["level", "java", "radar"]

Sample output: 2

220. Find the longest palindrome word

Filter palindromes before selecting the longest item.

```
Optional<String> longestPalindrome = words.stream()
    .filter(w -> w.equalsIgnoreCase(new StringBuilder(w).reverse().toString()))
    .max(Comparator.comparingInt(String::length));
```

Sample input: words = ["level", "java", "radar"]

Sample output: "level"

221. Find the shortest palindrome word

Filter palindromes before selecting the shortest item.

```
Optional<String> shortestPalindrome = words.stream()
    .filter(w -> w.equalsIgnoreCase(new StringBuilder(w).reverse().toString()))
    .min(Comparator.comparingInt(String::length));
```

Sample input: words = ["level", "java", "radar"]

Sample output: "java"

222. Find the largest anagram group

Group first, then select the group with the most words.

```
Map<String, List<String>> groups = words.stream().collect(Collectors.groupingBy(w ->
    w.toLowerCase(Locale.ROOT).chars().sorted().mapToObj(c -> String.valueOf((char) c))
    .collect(Collectors.joining())));
Optional<List<String>> largest = groups.values().stream().max(Comparator.comparingInt(List::size));
```

Sample input: words = ["eat", "tea", "ate", "tan"]

Sample output: ["eat", "tea", "ate"]

223. Count anagram groups

Count only groups that contain at least two anagrams.

```
Map<String, List<String>> groups = words.stream().collect(Collectors.groupingBy(w ->
    w.toLowerCase(Locale.ROOT).chars().sorted().mapToObj(c -> String.valueOf((char) c))
    .collect(Collectors.joining())));
long groupCount = groups.values().stream().filter(group -> group.size() > 1).count();
```

Sample input: words = ["eat", "tea", "ate", "tan"]

Sample output: 1

224. Find anagrams of a target word

Compare each word's canonical sorted-letter signature with the target's signature.

```
String key = target.toLowerCase(Locale.ROOT).chars().sorted()
    .mapToObj(c -> String.valueOf((char) c)).collect(Collectors.joining());
List<String> matches = words.stream().filter(w -> w.toLowerCase(Locale.ROOT).chars().sorted()
    .mapToObj(c -> String.valueOf((char) c)).collect(Collectors.joining()).equals(key))
    .collect(Collectors.toList());
```

Sample input: target = "eat"; words = ["eat", "tea", "tan"]

Sample output: ["eat", "tea"]

225. Compare two strings exactly

Use equals for a case-sensitive comparison.

```
boolean equal = first.equals(second);
```

Sample input: first = "java"; second = "Java"

Sample output: false

226. Check two strings have the same distinct characters

Compare the two character sets, ignoring duplicates and order.

```
Set<Integer> left = first.chars().boxed().collect(Collectors.toSet());
Set<Integer> right = second.chars().boxed().collect(Collectors.toSet());
boolean sameCharacters = left.equals(right);
```

Sample input: first = "abb"; second = "ba"

Sample output: true

227. List common characters between two strings

Keep distinct characters from the first string that occur in the second.

```
List<Character> common = first.chars().distinct().filter(c -> second.indexOf(c) >= 0)
    .mapToObj(c -> (char) c).collect(Collectors.toList());
```

Sample input: first = "hello"; second = "world"

Sample output: [l, o]

228. List characters only in the first string

Exclude all first-string characters that occur in the second.

```
List<Character> onlyFirst = first.chars().distinct().filter(c -> second.indexOf(c) < 0)
    .mapToObj(c -> (char) c).collect(Collectors.toList());
```

Sample input: first = "hello"; second = "world"

Sample output: [h, e]

229. List characters only in the second string

Exclude all second-string characters that occur in the first.

```
List<Character> onlySecond = second.chars().distinct().filter(c -> first.indexOf(c) < 0)
    .mapToObj(c -> (char) c).collect(Collectors.toList());
```

Sample input: first = "hello"; second = "world"

Sample output: [w, r, d]

230. Find different words in two sentences

Compute the symmetric difference of normalized word sets.

```
Set<String> left = Arrays.stream(first.toLowerCase(Locale.ROOT).split("\\s+")).collect(Collectors.toSet());
Set<String> right = Arrays.stream(second.toLowerCase(Locale.ROOT).split("\\s+")).collect(Collectors.toSet());
Set<String> different = Stream.concat(left.stream().filter(w -> !right.contains(w)),
    right.stream().filter(w -> !left.contains(w)))
    .collect(Collectors.toSet());
```

Sample input: first = "Java API"; second = "API Spring"

Sample output: set contents (order may vary): "java", "spring"

231. Intersect two string arrays

Return distinct values that occur in both arrays.

```
Set<String> right = Arrays.stream(secondArray).collect(Collectors.toSet());
List<String> intersection = Arrays.stream(firstArray).filter(right::contains).distinct()
    .collect(Collectors.toList());
```

Sample input: firstArray = ["java", "api", "stream"]; secondArray = ["api", "spring"]

Sample output: ["api"]

232. Union two string arrays

Concatenate, then remove duplicates.

```
List<String> union = Stream.concat(Arrays.stream(firstArray), Arrays.stream(secondArray))
    .distinct().collect(Collectors.toList());
```

Sample input: firstArray = ["java", "api", "stream"]; secondArray = ["api", "spring"]

Sample output: ["java", "api", "stream", "spring"]

233. Keep strings shorter than N

Complement the existing greater-than filter with a strict upper bound.

```
List<String> result = words.stream()
    .filter(s -> s.length() < n).collect(Collectors.toList());
```

Sample input: words = ["a", "bb", "ccc"]; n = 3

Sample output: ["a", "bb"]

234. Keep only alphanumeric strings

Validate that every character is a letter or digit.

```
List<String> result = values.stream()
    .filter(s -> s.matches("[A-Za-z0-9]+"))
    .collect(Collectors.toList());
```

Sample input: values = ["a1", "abc", "123", "a-b"]

Sample output: ["a1", "abc", "123"]

235. Keep strings with the same starting and ending character

Exclude empty values before comparing endpoints.

```
List<String> result = words.stream().filter(s -> !s.isEmpty())
    .filter(s -> s.charAt(0) == s.charAt(s.length() - 1))
    .collect(Collectors.toList());
```

Sample input: words = ["level", "java", "aba"]

Sample output: ["level", "aba"]

236. Sort alphabetically then by length

Use lexical order first and length only as the tie-breaker.

```
List<String> sorted = words.stream()
    .sorted(Comparator.<String>naturalOrder().thenComparingInt(String::length))
    .collect(Collectors.toList());
```

Sample input: words = ["java", "api", "stream"]

Sample output: ["api", "java", "stream"]

237. Sort words by vowel count

Use the number of vowels as the comparator key.

```
List<String> sorted = words.stream().sorted(Comparator.comparingLong(w ->
    w.toLowerCase(Locale.ROOT).chars().filter(c -> "aeiou".indexOf(c) >= 0).count()))
    .collect(Collectors.toList());
```

Sample input: words = ["java", "aaa", "bbb"]

Sample output: ["bbb", "java", "aaa"]

238. Sort words by consonant count

Use the number of consonants as the comparator key.

```
List<String> sorted = words.stream().sorted(Comparator.comparingLong(w ->
    w.toLowerCase(Locale.ROOT).chars().filter(Character::isLetter)
        .filter(c -> "aeiou".indexOf(c) < 0).count()))
    .collect(Collectors.toList());
```

Sample input: words = ["java", "aaa", "bbb"]

Sample output: ["aaa", "java", "bbb"]

239. Lowercase the first character

Transform only the first character of non-empty strings.

```
List<String> result = words.stream().filter(s -> !s.isEmpty())
    .map(s -> s.substring(0, 1).toLowerCase(Locale.ROOT) + s.substring(1))
    .collect(Collectors.toList());
```

Sample input: words = ["Java", "API"]

Sample output: ["java", "aPI"]

240. Convert a Character list to a String

Map Character values to text, then join them.

```
String result = characters.stream()
    .map(String::valueOf).collect(Collectors.joining());
```

Sample input: characters = [J, a, v, a]

Sample output: "Java"

241. Flatten all characters across a string list

Flatten character streams without removing duplicate characters.

```
List<Character> characters = words.stream().flatMapToInt(String::chars)
    .mapToObj(c -> (char) c).collect(Collectors.toList());
```

Sample input: words = ["java", "api"]

Sample output: [j, a, v, a, a, p, i]

242. Find duplicate characters across a string list

Flatten all characters, count them, and retain repeated keys.

```
Map<Character, Long> counts = words.stream().flatMapToInt(String::chars).mapToObj(c -> (char) c)
    .collect(Collectors.groupingBy(Function.identity(), Collectors.counting()));
List<Character> duplicates = counts.entrySet().stream().filter(e -> e.getValue() > 1)
    .map(Map.Entry::getKey).collect(Collectors.toList());
```

Sample input: words = ["java", "api", "java"]

Sample output: [a]

243. Count characters across a string list

Build a cross-string character-frequency map.

```
Map<Character, Long> counts = words.stream().flatMapToInt(String::chars).mapToObj(c -> (char) c)
    .collect(Collectors.groupingBy(Function.identity(), Collectors.counting()));
```

Sample input: words = ["java", "api"]

Sample output: map contents (entry order may vary): j -> 1, a -> 3, v -> 1, p -> 1, i -> 1

244. Count words across multiple sentences

Flatten sentence tokens, then count the resulting stream.

```
long count = sentences.stream().flatMap(s -> Arrays.stream(s.trim().split("\\s+")))
    .filter(s -> !s.isEmpty()).count();
```

Sample input: sentences = ["Java is powerful", "Stream API is useful"]

Sample output: 7

245. Find the most frequent word across sentences

Flatten, normalize, count, and maximize frequency.

```
Optional<String> mostFrequent = sentences.stream()
    .flatMap(s -> Arrays.stream(s.toLowerCase(Locale.ROOT).split("\\s+")))
    .collect(Collectors.groupingBy(Function.identity(), Collectors.counting())).entrySet().stream()
    .max(Map.Entry.comparingByValue()).map(Map.Entry::getKey);
```

Sample input: sentences = ["Java Java Java", "API is useful"]

Sample output: "java"

246. Find the longest word across sentences

Flatten all sentence tokens before taking the maximum length.

```
Optional<String> longest = sentences.stream().flatMap(s -> Arrays.stream(s.trim().split("\\s+")))
    .filter(s -> !s.isEmpty()).max(Comparator.comparingInt(String::length));
```

Sample input: sentences = ["Java Stream", "API"]

Sample output: "Stream"

247. Count characters with toMap

Use a merge function to accumulate character counts.

```
Map<Character, Long> counts = text.chars().mapToObj(c -> (char) c)
    .collect(Collectors.toMap(Function.identity(), c -> 1L, Long::sum, LinkedHashMap::new));
```

Sample input: text = "AAABBCC"

Sample output: A -> 3, B -> 2, C -> 2

248. Group words by vowel count

Bucket words by their number of vowels.

```
Map<Integer, List<String>> byVowels = words.stream().collect(Collectors.groupingBy(w ->
    (int) w.toLowerCase(Locale.ROOT).chars().filter(c -> "aeiou".indexOf(c) >= 0).count()));
```

Sample input: words = ["java", "aaa", "bbb"]

Sample output: map contents (entry order may vary): 2 -> ["java"], 3 -> ["aaa"], 0 -> ["bbb"]

249. Group words by their frequency

Build a frequency map, then invert it into count-to-words groups.

```
Map<String, Long> counts = words.stream()
    .collect(Collectors.groupingBy(Function.identity(), Collectors.counting()));
Map<Long, List<String>> byFrequency = counts.entrySet().stream()
    .collect(Collectors.groupingBy(Map.Entry::getValue,
    Collectors.mapping(Map.Entry::getKey, Collectors.toList())));
```

Sample input: words = ["java", "java", "api"]

Sample output: map contents (entry order may vary): 2 -> ["java"], 1 -> ["api"]

250. Partition palindrome and non-palindrome words

Use a palindrome predicate with partitioningBy.

```
Map<Boolean, List<String>> partition = words.stream().collect(Collectors.partitioningBy(w ->
    w.equalsIgnoreCase(new StringBuilder(w).reverse().toString())));
```

Sample input: words = ["level", "java", "radar"]

Sample output: true -> ["level", "radar"], false -> ["java"]

251. Partition words by even or odd length

The true group contains even-length words.

```
Map<Boolean, List<String>> partition = words.stream()
    .collect(Collectors.partitioningBy(w -> w.length() % 2 == 0));
```

Sample input: words = ["java", "api", "cat"]

Sample output: true -> ["java"], false -> ["api", "cat"]

252. Partition empty and non-empty strings

The true group contains empty values.

```
Map<Boolean, List<String>> partition = words.stream()
    .collect(Collectors.partitioningBy(String::isEmpty));
```

Sample input: words = ["", "java", ""]

Sample output: true -> ["", ""], false -> ["java"]

253. Partition vowel-start and consonant-start words

The true group contains non-empty words starting with a vowel.

```
Map<Boolean, List<String>> partition = words.stream().filter(w -> !w.isEmpty())
    .collect(Collectors.partitioningBy(w -> "aeiouAEIOU".indexOf(w.charAt(0)) >= 0));
```

Sample input: words = ["apple", "java", "code"]

Sample output: true -> ["apple"], false -> ["java", "code"]

254. Partition strings by digit presence

The true group contains strings with at least one digit.

```
Map<Boolean, List<String>> partition = values.stream()
    .collect(Collectors.partitioningBy(s -> s.chars().anyMatch(Character::isDigit)));
```

Sample input: values = ["a1", "java", "2b"]

Sample output: true -> ["a1", "2b"], false -> ["java"]

255. Find the maximum-frequency character with stable tie-breaking

LinkedHashMap preserves first occurrence when counts tie.

```
Map<Character, Long> counts = text.chars().mapToObj(c -> (char) c)
    .collect(Collectors.groupingBy(Function.identity(), LinkedHashMap::new, Collectors.counting()));
Optional<Character> mostFrequent = counts.entrySet().stream()
    .max(Comparator.comparingLong(Map.Entry::getValue)).map(Map.Entry::getKey);
```

Sample input: text = "AAABBCCDDD"

Sample output: "A"

256. Check two strings have identical character frequencies

Compare their complete character-frequency maps.

```
Map<Character, Long> left = first.chars().mapToObj(c -> (char) c)
    .collect(Collectors.groupingBy(Function.identity(), Collectors.counting()));
Map<Character, Long> right = second.chars().mapToObj(c -> (char) c)
    .collect(Collectors.groupingBy(Function.identity(), Collectors.counting()));
boolean sameFrequencies = left.equals(right);
```

Sample input: first = "aabb"; second = "bbaa"

Sample output: true

257. Map each word to its length

Produce a word-to-length map; keep the first value if a word repeats.

```
Map<String, Integer> lengths = Arrays.stream(sentence.trim().split("\\s+"))
    .collect(Collectors.toMap(Function.identity(), String::length, (first, ignored) -> first));
```

Sample input: sentence = "Java is Java"

Sample output: map contents (entry order may vary): Java -> 4, is -> 2